DISEÑO DE ALGORITMOS

ASESOR:
ING. GERARDO ACEVEDO VEGA

UNIDAD 4 · ACTIVIDAD COMPLEMENTARIA 1

ANÁLISIS FORMAL: PRECONDICIONES Y POSTCONDICIONES

10 DE NOVIEMBRE DE 2026

GRUPO 8191 · INFORMÁTICA

Alumno: Mauro Noel Sánchez Campos

Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Introducción

Antes de diseñar un algoritmo conviene definir con precisión qué debe cumplirse antes de ejecutarlo (precondiciones) y qué debe ser cierto al terminar (postcondiciones). Este análisis formal evita ambigüedades y sirve después para diseñar las pruebas. En esta actividad lo aplico al problema de un cajero automático que permite retirar efectivo.

Problema: el usuario indica el monto que desea retirar; el sistema verifica que tenga saldo suficiente y que el monto sea múltiplo de 50; si todo es correcto entrega el efectivo y muestra el nuevo saldo; si no, muestra el error correspondiente.

1. Objetivo del problema

Permitir que un usuario retire efectivo de su cuenta solo cuando el monto solicitado sea válido (positivo, múltiplo de 50 y no mayor que su saldo), actualizando el saldo y mostrándolo, o informando con un mensaje claro la causa por la que no se pudo realizar el retiro.

2. Datos de entrada y precondiciones

Dato de entrada	Tipo	Precondición
saldo	Real	Es el saldo actual de la cuenta, obtenido del sistema del banco. Se cumple que saldo ≥ 0. El usuario ya está autenticado (tarjeta y NIP válidos).
monto	Entero	Es un número capturado por el usuario. Antes de procesarlo no se garantiza que sea válido: puede ser negativo, cero, no múltiplo de 50 o mayor que el saldo; por eso el algoritmo debe validarlo.

3. Datos de salida y postcondiciones

Dato de salida	Tipo	Postcondición
efectivo entregado	Entero	Si el retiro es válido: efectivo = monto. Si no es válido: no se entrega efectivo (efectivo = 0).
nuevoSaldo	Real	Si el retiro es válido: nuevoSaldo = saldo − monto, y nuevoSaldo ≥ 0. Si no es válido: el saldo no cambia (nuevoSaldo = saldo).
mensaje	Cadena	Si es válido: "Retiro exitoso. Nuevo saldo: $…". Si no: el mensaje de error que corresponde a la primera regla que no se cumplió.

4. Restricciones

- El monto debe ser mayor que 0.
- El monto debe ser múltiplo de 50 (monto mod 50 = 0), porque el cajero solo tiene billetes que permiten armar cantidades de 50 en 50.
- El monto no puede ser mayor que el saldo disponible (monto ≤ saldo).
- El orden de validación es: primero monto positivo, después múltiplo de 50 y al final saldo suficiente, para mostrar un solo mensaje claro.
- Una operación no válida no debe modificar el saldo.

5. Tabla de casos de prueba

Para todos los casos se supone un saldo inicial de $1,000.00.

Caso	Tipo	Monto	Salida esperada
1	Válido	$500	Entrega $500. Mensaje: "Retiro exitoso. Nuevo saldo: $500.00".
2	Válido (límite: retira todo el saldo)	$1,000	Entrega $1,000. Mensaje: "Retiro exitoso. Nuevo saldo: $0.00".
3	Inválido: monto > saldo	$1,050	No entrega efectivo. Mensaje: "Saldo insuficiente". Saldo sin cambios: $1,000.00.
4	Inválido: no múltiplo de 50	$120	No entrega efectivo. Mensaje: "El monto debe ser múltiplo de 50". Saldo sin cambios.
5	Inválido: monto negativo	−$100	No entrega efectivo. Mensaje: "El monto debe ser mayor que cero". Saldo sin cambios.

El caso 2 es un caso límite importante: comprueba que la condición sea monto ≤ saldo y no monto < saldo, porque retirar exactamente todo el saldo sí está permitido.

Algoritmo en pasos numerados

Para comprobar que el análisis es completo, este algoritmo, escrito como una secuencia de pasos numerados, cumple todas las precondiciones y postcondiciones anteriores:

1. Inicio.

2. Leer el monto que desea retirar el usuario.
3. Si el monto es menor o igual a 0, mostrar "El monto debe ser mayor que cero" e ir al paso 7.
4. Si el residuo de monto entre 50 es distinto de 0, mostrar "El monto debe ser múltiplo de 50" e ir al paso 7.
5. Si el monto es mayor que el saldo, mostrar "Saldo insuficiente" e ir al paso 7.
6. Calcular saldo ← saldo − monto, entregar el efectivo y mostrar "Retiro exitoso. Nuevo saldo: " y el saldo.
7. Fin.

Conclusión

Definir precondiciones y postcondiciones me obligó a pensar en todos los casos antes de escribir una sola línea del algoritmo, sobre todo en los que fallan. En un sistema bancario esto es crítico: una validación en el orden equivocado o un "<" en lugar de "≤" puede impedir un retiro legítimo o, peor, permitir uno que deje el saldo negativo. La tabla de casos convierte el análisis directamente en el plan de pruebas.

Referencias

Cairó, O. (2005). *Metodología de la programación: algoritmos, diagramas de flujo y programas* (3a ed.). Alfaomega.

Joyanes Aguilar, L. (2020). *Fundamentos de programación: algoritmos, estructuras de datos y objetos* (5a ed.). McGraw-Hill Interamericana.

Kanagusico Hernández, E. D., Moreno García, R. C., y Hernández Sánchez, E. (2026). *Diseño de algoritmos* [Apunte digital, Licenciatura en Informática]. Universidad Nacional Autónoma de México, Facultad de Contaduría y Administración.